GNN Code Explanation Report

Report: Explanation of Provided GNN Code

Overview

This report explains the provided Python code, which builds and trains a small Graph Neural Network (GNN) using GraphSAGE from the torch_geometric library. The purpose is to classify nodes in a graph as either benign or malicious.

1. Installing Dependencies

The code begins by installing torch_geometric, a library designed for graph machine learning tasks. This toolkit extends PyTorch with graph operations and GNN layers.

2. Importing Required Libraries

The script imports PyTorch, GraphSAGE convolution layers, and functional utilities for activation functions and loss computation.

3. Constructing the Graph

Node Features:

The code defines a small graph of six nodes. Each node is represented by a 2■dimensional feature vector. Some nodes are benign, and others are malicious. The feature vectors express simple characteristics for learning.

Edges:

Edges are pairs of connected nodes. The graph is undirected, so connections appear in both directions. Structure:

- Nodes 0, 1, and 2 form a benign cluster

- Nodes 3, 4, and 5 form another cluster with malicious nodes

Labels:

Each node receives a label:

0 → benign

1 → malicious

4. Defining the GraphSAGE Model

The model includes:

- Input layer (2 features)

- Hidden layer (4 units)

- Output layer (2 classes)

- ReLU activation

- log_softmax for classification

GraphSAGE learns node embeddings by aggregating information from neighbors.

5. Training the Model

Training uses:

- NLLLoss

- Adam optimizer

- 50 epochs

Each epoch performs:

Forward pass → loss → backpropagation → weight update.

6. Predictions

After training, the model predicts labels for all nodes. Expected behavior:

- Benign nodes have similar predictions

- Malicious nodes are identified through features and connectivity

Conclusion

The code provides a full pipeline for building and training a GNN for node classification, demonstrating how GraphSAGE detects malicious patterns using graph structure.